

# Course Outline: CLIs for AI — Building Tools Your Agent Can Call

---

**Title:** CLIs for AI — Building Tools Your Agent Can Call **Skill:** \_\_ — Skill: Proveer de herramientas a un Agente mediante CLIs **Learnpack:** + CLIs y la IA **File:** s\_\_-w20d58-clis-y-la-ia **Prerequisite:** CLIs — The Developer's Best Tool (what CLIs are, Click basics) **Target Audience:** Beginner developers who just built their first CLI and want to understand how AI agents use CLIs **Total Lessons:** 8 **Format:** Mixed (12.5% hands-on)

---

## Course Description

The previous learnpack taught you how to build CLIs for human users. This course answers a different question: what happens when the user is an AI agent, not a human? AI agents call CLIs via code — they read the output programmatically, check exit codes, and parse the results to make decisions. This changes what "good" means: an AI-friendly CLI needs structured output (JSON over prose), predictable exit codes, and clear argument documentation. Students finish with a working Python CLI designed specifically to be called by an AI agent.

---

## Course Philosophy

This course emphasizes:

- The key insight that AI agents and humans have different needs from CLIs
  - Concrete, observable differences: JSON vs prose, exit codes, schema documentation
  - Practical skills — the student builds something an AI could actually call
- 

## Course Statistics Summary

| Section                     | Lessons  |
|-----------------------------|----------|
| 00 - Welcome                | 1        |
| 01 - Why AI Uses CLIs       | 2        |
| 02 - Building AI-Ready CLIs | 3        |
| 03 - Assessment             | 1        |
| 04 - Outro                  | 1        |
| <b>Total</b>                | <b>8</b> |

---

00.0 CLIs as AI Tools 

**Learning Objectives:**

- Explain how AI agents use CLIs as part of their tool ecosystem
- Describe the fundamental difference between a human-facing CLI and an AI-facing CLI

### Content Outline:

- Recap: you just built a CLI that a human runs and reads. Now what about AI?
- How AI agents work: they receive a task, decide which tool to use, call the tool, read the output, and act on it
- CLI tools fit perfectly into this loop — they're already standardized (stdin/stdout/stderr, exit codes)
- The key difference: humans tolerate prose; agents need structure — a human can parse "Error: file not found in /data" but an agent needs `{"error": "file_not_found", "path": "/data"}`
- What this course covers: agent architecture, why agents use CLIs, and how to build one they can reliably call

**Transitions:** This lesson sets the context. Section 01 dives into how agents actually invoke CLIs and what they do with the output.

### Key Principles:

- AI agents are just code — they read strings, parse JSON, and check numbers
- A CLI that's good for humans can be terrible for agents (and vice versa)

### Examples:

- Human-friendly output: `Successfully processed 47 records. 3 were skipped due to missing values.`
- Agent-friendly output: `{"processed": 47, "skipped": 3, "reason": "missing_values"}`  
The human reads the first and understands it. The agent can parse the second without any guessing.

## 01.0 Why AI Uses CLIs

### Learning Objectives:

- Explain why CLIs are a natural tool interface for AI agents
- Describe the agent tool-calling loop: receive task → select tool → call via CLI → parse output → act
- Preview what lesson 01.1 covers (the mechanics of how agents invoke CLIs)

### Content Outline:

- **Why agents use CLIs:** CLIs are universal — any language, any system, any environment can run a command and read output
- **The agent tool-calling loop:**
  1. Agent receives a task: "What is the row count of sales.csv?"
  2. Agent selects a tool: `data-tool count --file sales.csv`
  3. Agent calls the CLI: runs the command via subprocess

4. Agent reads the output: parses the JSON response
  5. Agent acts: reports the count or passes it to the next step
- **Why not a direct API?** CLIs are simpler to integrate, don't require authentication setup, and work in any environment without SDK dependencies
  - **Why not just write Python code?** Reusing existing CLI tools means not reinventing everything — an agent that can use `git`, `curl`, `jq`, and your custom tools immediately has enormous capability
  - Preview: 01.1 covers the subprocess call and how agents read exit codes and output

**Transitions:** Now you know WHY. Lesson 01.1 shows exactly HOW an agent calls a CLI — the subprocess mechanics and what it reads.

### Key Principles:

- CLIs are the universal interface: any tool that can be invoked from the terminal can be used by an agent
- Agents don't just run CLIs — they run them, read the result, and branch based on what they find

**Examples:** An agent helping with data analysis:

1. `data-tool validate --file sales.csv` → check if file is valid
  2. `data-tool count --file sales.csv` → get row count
  3. `data-tool summarize --file sales.csv --columns revenue,quantity` → get statistics
- Each step informs the next.
- 

## 01.1 How AI Invokes CLIs

### Learning Objectives:

- Describe how Python code calls a CLI using `subprocess`
- Explain what an agent reads from a CLI call: stdout, stderr, and exit code
- Identify why exit codes are critical for agent error handling

### Content Outline:

- **The subprocess pattern:** how agents call CLIs in Python

```
import subprocess
result = subprocess.run(
    ["data-tool", "count", "--file", "sales.csv"],
    capture_output=True,
    text=True
)
output = result.stdout           # what the CLI printed
error = result.stderr           # what the CLI printed to error
exit_code = result.returncode   # 0 = success, non-zero = failure
```

- **Stdout:** the main output — the agent reads and parses this

- **Stderr:** error messages — agents check this when something goes wrong
- **Exit code:** the critical signal — 0 means success, non-zero means failure; agents branch on this
  - If `exit_code == 0`: parse the output and proceed
  - If `exit_code != 0`: handle the error (retry, report, stop)
- **JSON parsing:** after reading stdout, agents parse it as JSON

```
import json
data = json.loads(result.stdout)
row_count = data["count"]
```

**Transitions:** You know how agents call CLIs and what they read. Section 02 focuses on building CLIs that work well for this pattern.

### Key Principles:

- Exit code 0 = "I did my job"; any other code = "something went wrong" — agents **MUST** check this
- Agents parse stdout as structured data — your CLI must output something parseable, not human prose

### Anti-patterns:

- **Printing progress messages to stdout:** agents parse stdout as data; progress messages break JSON parsing
  - Fix: print progress to stderr, data to stdout
- **Exit code 0 even on errors:** an agent that always gets exit code 0 cannot detect failures
  - Fix: always exit with non-zero code on error

### Examples:

```
# Agent calling a CLI and handling the result
result = subprocess.run(["my-tool", "--format", "json"], capture_output=True,
text=True)

if result.returncode != 0:
    print(f"Tool failed: {result.stderr}")
    # handle the error
else:
    data = json.loads(result.stdout)
    # use the data
```

---

## 02.0 Building AI-Ready CLIs

### Learning Objectives:

- List the key differences between a human-friendly CLI and an AI-friendly CLI

- Apply three rules to make a CLI agent-ready: JSON output, correct exit codes, clear argument documentation
- Preview what lessons 02.1 (AI-friendly principles) and 02.2 (hands-on) cover

### Content Outline:

- **The core difference:** humans read text; agents parse data
- **Rule 1: Structured output (JSON)**
  - ALL output goes as JSON to stdout
  - On success: `{"status": "ok", "result": {...}}`
  - On error: `{"status": "error", "message": "..."}`
  - Never mix JSON with prose in stdout
- **Rule 2: Correct exit codes**
  - 0 = success (agent can trust the output)
  - 1 = general error (agent should check stderr/message)
  - Other codes: use specific codes for specific failures when needed
- **Rule 3: Clear, predictable argument interface**
  - Use explicit flags instead of positional arguments (agents build command strings programmatically)
  - Every argument must be documented (agents read `--help` to understand the interface)
  - Consistent naming: `--input-file` not `--f` or `--file_path`
- **Progress vs data:** send data to stdout, progress/logs to stderr

**Transitions:** These are the rules. Lesson 02.1 goes deeper into each principle and shows examples of good vs bad AI-CLI design.

### Key Principles:

- An AI-ready CLI is not less useful to humans — JSON output plus a `--human` flag for prose makes everyone happy
- The discipline of AI-ready CLIs improves ALL CLI design: structured output, correct errors, clear arguments

**Examples:** Comparing human-ready vs AI-ready for the same operation:

Human-ready: `Successfully loaded 500 rows from data.csv` AI-ready: `{"status": "ok", "rows_loaded": 500, "file": "data.csv"}`

---

## 02.1 AI-Friendly CLI Principles

### Learning Objectives:

- Design a CLI's output schema that an agent can rely on
- Apply the `--format` flag pattern to serve both human and AI consumers
- Explain why argument predictability matters more for agents than for humans

### Content Outline:

- **Output schema design:** define exactly what JSON structure your CLI always returns

- Success: `{"status": "ok", "data": {...}}`
- Error: `{"status": "error", "code": "...", "message": "..."}"`
- The schema must be stable — agents depend on field names; changing them breaks integrations
- **The `--format` flag pattern:** one CLI, two audiences

```
my-tool --format json → {"status": "ok", "count": 47}
my-tool --format human → Processed 47 records successfully.
```

Default `--format` is json for agent-friendly defaults

- **Argument predictability:**
  - Agents build argument lists in code: `["--file", filename, "--mode", "validate"]`
  - Named flags (not positional) are safer: the agent doesn't need to remember order
  - Avoid required arguments with no default — agents need to know what to provide
- **Idempotency:** an agent might call your CLI multiple times; the CLI should handle this gracefully
- **Timeouts and long-running operations:** if your CLI can take a long time, add a `--timeout` argument

**Transitions:** With these principles clear, lesson 02.2 builds a complete AI-ready CLI tool.

### Key Principles:

- An agent that can't parse your output can't use your tool — schema consistency is non-negotiable
- Design for the programmatic caller first; human friendliness is a feature you add on top

### Anti-patterns:

- **Dynamic field names:** `{"result_2024": 47}` — agents can't reliably find a field if its name changes
- **Mixing success and error JSON structures:** agents need to know what success looks like vs. failure

**Examples:** Good output schema design:

```
// Success
{
  "status": "ok",
  "operation": "count",
  "result": {"rows": 500, "columns": 8}
}

// Error
{
  "status": "error",
  "code": "file_not_found",
```

```
"message": "Could not open sales.csv: file does not exist"
}
```

The agent checks `status` first, then acts accordingly.

---

## 02.2 A CLI for Your Agent

**Challenge Prompt:** Build a Python CLI using Click that accepts a `--file` option and returns a JSON response with the file's existence status and size. Return exit code 0 on success and exit code 1 if the file does not exist.

### Solution Code:

```
import click
import json
import os
import sys

@click.command()
@click.option("--file", required=True, help="Path to the file to inspect.")
def inspect(file):
    if not os.path.exists(file):
        result = {"status": "error", "code": "file_not_found", "message":
f"File not found: {file}"}
        click.echo(json.dumps(result))
        sys.exit(1)

    size = os.path.getsize(file)
    result = {"status": "ok", "file": file, "size_bytes": size, "exists":
True}
    click.echo(json.dumps(result))

if __name__ == "__main__":
    inspect()
```

### Challenge Code:

```
import click
import json
import os
import sys

@click.command()
@click.option("--file", required=True, help="Path to the file to inspect.")
def inspect(file):
    # Step 1: if the file doesn't exist, output a JSON error and exit with
code 1
```

```

if not os.path.exists(file):
    # your code here
    pass

# Step 2: if the file exists, output JSON with status="ok", file path,
and size in bytes
size = os.path.getsize(file)
# your code here

if __name__ == "__main__":
    inspect()

```

## 03.0 CLI for AI Assessment 🧠

### Learning Objectives:

- Demonstrate understanding of how agents use CLIs
- Correctly identify AI-friendly vs human-friendly CLI design choices
- Apply the JSON output + exit code pattern correctly

### Question Format: Multiple choice

### Questions:

1. An AI agent runs a CLI command and gets exit code 0 with output `{"status": "error", "message": "file not found"}`. What is the most likely problem?
  - a) The agent is not installed correctly
  - b) The CLI is not outputting valid JSON
  - c) The CLI returned exit code 0 for an error — the agent cannot detect the failure
  - d) The agent should check stderr instead
2. Which subprocess call correctly captures both stdout and stderr from a CLI?
  - a) `subprocess.run(cmd)`
  - b) `subprocess.run(cmd, capture_output=True, text=True)`
  - c) `subprocess.call(cmd, shell=True)`
  - d) `os.system(cmd)`
3. A CLI outputs: `Successfully loaded 500 rows from data.csv` to stdout. What is wrong with this for an AI agent?
  - a) Nothing — agents can read English text
  - b) The message is too long
  - c) Agents parse stdout as structured data; prose output breaks JSON parsing
  - d) The message should be sent to a log file
4. What is the correct exit code for a successful CLI operation?
  - a) 1

- b) -1
- c) 0
- d) 200

5. Which output schema is better designed for an AI agent?

- a) `Processed: 500 rows. Errors: 3.`
- b) `{"processed": 500, "errors": 3}`
- c) Both are equally good — agents handle both
- d) Neither — agents should use APIs, not CLIs

6. Why is the `--format` flag pattern useful for AI-friendly CLIs?

- a) It makes the CLI faster
- b) It allows the same CLI to serve both human and agent consumers without duplicating code
- c) It encrypts the output for security
- d) It is required by all modern CLI frameworks

7. An agent builds the command `["my-tool", "output.csv", "validate"]` where the order of positional arguments matters. What risk does this create?

- a) No risk — positional arguments are always correct
- b) If the argument order changes in the CLI, the agent's call breaks silently
- c) Positional arguments produce binary output that agents can't parse
- d) The agent needs root access for positional arguments

### Evaluation Criteria:

- 7/7: Excellent — strong understanding of agent-CLI interaction
- 5–6/7: Good — minor gaps in exit code or output design
- 3–4/7: Needs review — revisit Sections 01 and 02
- Below 3/7: Restart recommended

**Score Interpretation:** Questions map to: exit code confusion (Q1), subprocess pattern (Q2), prose output problem (Q3), exit code 0 (Q4), JSON vs prose (Q5), `--format` flag (Q6), positional arg risk (Q7).

---

## 04.0 Your Agent Has Tools

### Content Outline:

- Course recap: what students accomplished
  - Understood why AI agents use CLIs as their primary interface to external tools
  - Traced the agent tool-calling loop: receive task → select tool → call via subprocess → parse output → act
  - Learned the three rules of AI-ready CLIs: JSON output, correct exit codes, clear argument interface
  - Designed an output schema that agents can rely on

- Built a Python CLI that returns structured JSON and correct exit codes — ready to be called by any agent
- Connection to bigger picture: this course opened the door to the MCP (Model Context Protocol) learnpacks that follow — MCPs are the next evolution of how agents access tools, built on the same principles
- Next steps: the next learnpacks introduce MCPs — a standardized protocol that goes beyond CLI calls to provide richer agent-tool interaction with type safety and schema validation
- Resources for further exploration:
  - Anthropic tool use documentation
  - OpenAI function calling documentation
  - LangChain tools documentation
  - Click documentation for advanced CLI patterns
- Encouragement: you've just bridged the gap between "I write code" and "my code works with AI" — that is a genuinely powerful position to be in

**Note:** This is conclusion only — no theory, exercises, or assessment.

---